



University of Asia Pacific
Department of Computer Science and
Engineering

Compiler Design
(CSE 430)

Submitted By

Mazharul Islam Sourav

ID: 22101133

Section: C-2

Semester: 4-2

Submitted To

Md Shafayatul Haque

Lecturer

Department of Computer Science
and Engineering

University of Asia Pacific

Work 01: Write a program for lexical analysis.

Code -

```
import re

KEYWORDS = {
    'if', 'else', 'for', 'while', 'do', 'int', 'float',
    'char', 'void', 'return', 'break', 'continue',
    'double', 'switch', 'case', 'default'
}

def remove_comments(code):

    code = re.sub(r'\/\*.*?\*/', '', code, flags=re.DOTALL)
    code = re.sub(r'//.*', '', code)

    return code

def lexical_analyzer(code):

    tokens = {
        'Keyword': set(),
        'Identifier': set(),
        'Constant': set(),
        'Arithmetic Operator': set(),
        'Logical Operator': set(),
        'Punctuation': set(),
        'Parenthesis': set()
    }

    char_literals = re.findall(r'"(?:\\.|[^\"])*"', code)
    string_literals = re.findall(r'"(?:\\.|[^\"])*"', code)
    numeric_constants = re.findall(r'\d+\.\d+|\d+', code)

    tokens['Constant'].update(char_literals)
    tokens['Constant'].update(string_literals)
```

```

tokens['Constant'].update(numeric_constants)

# Remove string and char literals so their contents aren't
misclassified as identifiers or keywords.
code_no_literals = re.sub(r"'(?:\\.|[^\'])*'|\\"(?:\\.|[^\\""])*\"", ' ', code)

identifiers = re.findall(r'[a-zA-Z_][a-zA-Z0-9_]*', code_no_literals)

for word in identifiers:
    if word in KEYWORDS:
        tokens['Keyword'].add(word)
    else:
        tokens['Identifier'].add(word)

arithmetic =
re.findall(r'\+|\+|--|\+=|-=|\*=|/=|(?!<=>|=)(?!=)|\+|-|\*|/|%', code)
tokens['Arithmetic Operator'].update(arithmetic)
# '=' is classified here as an arithmetic operator (not assignment),
as per the output format given to us for this assignment.

logical = re.findall(r'&&|\||\||<=>|=|!=|>|<', code)
tokens['Logical Operator'].update(logical)

punctuation = re.findall(r'[;:,]', code)
tokens['Punctuation'].update(punctuation)

parenthesis = re.findall(r'[(){}\\[\\]]', code)
tokens['Parenthesis'].update(parenthesis)

return tokens

def read_file(filename):

```

```

try:
    with open(filename, 'r') as f:
        return f.read()
except:
    print(f"Error: Could not read '{filename}'. Please try again.")
    return None

def print_tokens(tokens):

    print("\nOutput--\n")

    for category, values in tokens.items():
        if values:
            print(f"{category} ({len(values)}): {'',
'.join(sorted(values))}")

def main():

    while True:
        filename = input("\nEnter input file name (example: input.txt): ")

        code = read_file(filename)

        if code is None:
            continue

        code = remove_comments(code)

        tokens = lexical_analyzer(code)

        print_tokens(tokens)

        again = input("\nDo you want to analyze another file? (y/n): ")
        .strip().lower()
        if again != 'y':
            print("Program sesh.")
            break

```

```
if __name__ == "__main__":  
    main()
```

Output -

```
Enter input file name (example: input.txt): input.txt  
  
Output--  
  
Keyword (3): if, int, void  
Identifier (4): a, b, c, main  
Constant (3): 0, 10, 2  
Arithmetic Operator (3): *, +, =  
Logical Operator (1): !  
Punctuation (2): ,, ;  
Parenthesis (4): (, ), {, }  
  
Do you want to analyze another file? (y/n): y  
  
Enter input file name (example: input.txt): input_2.txt  
  
Output--  
  
Keyword (4): char, float, int, void  
Identifier (6): a, b, c, ch, d, main  
Constant (3): 'a', 10, 3.14  
Arithmetic Operator (3): *, +, =  
Punctuation (2): ,, ;  
Parenthesis (4): (, ), {, }  
  
Do you want to analyze another file? (y/n): █
```

Code Link : [GitHub](#)